The Journal of Open Source Software

flory: A Python package for finding coexisting phases in multicomponent mixtures

Yicheng Qiang [1] **and David Zwicker** [1]

1 Max Planck Institute for Dynamics and Self-Organization, Göttingen, Germany

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor:

Submitted: 19 September 2024
Published: unpublished

License
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

DRAFT

Summary

Phase separation is an intrinsic property of mixtures that is widely observed in many scenarios, ranging from the simple demixing of oil and water to the condensation of biomolecules in cells. In multicomponent mixtures, phase separation can lead to many coexisting phases, which is crucial in many fields. To support such research, the `flory` package provides an easily accessible, performant, and extensible code that finds coexisting phases in multicomponent mixtures. The package expresses the free energy of the mixtures by several orthogonal aspects to cover a broad range of physical situations. In contrast to existing methods, the `flory` package implements a state-of-art method that is optimized for mixtures of many components.

Statement of need

Finding coexisting phases is a general task in many fields, such as chemical engineering (Lukas et al., 2007) and soft matter physics (Jacobs, 2023). There are several strategies to theoretically predict coexisting phases (Jacobs, 2023), including direct spatially-resolved simulations (Shrinivas & Brenner, 2021), the construction of the convex hull of the free energy (Mao et al., 2019), solving the balance equations (Zwicker & Laan, 2022), and direct minimization of the free energy (Lukas et al., 2007). However, there are only a few open-source packages that implement these strategies. Most notably, `pycalphad` (Otis & Liu, 2017) and `OpenCalphad` (Sundman et al., 2015) combine a database of candidate phases and several strategies above to compute phase diagrams (Calphad) of mixtures with few components. However, the computational cost of finding the coexisting phases scales strongly with component count, so systems with many different components cannot be analyzed efficiently with these packages.

The `flory` Python package is designed to fill this gap. It determines coexisting phases in a broad range of multicomponent mixtures while also being efficient enough for sampling entire phase diagrams when the composition is varied for many components.

Methods

The `flory` package is based on the free energy minimization strategy. To reduce computation cost, the package focuses on coexisting phases in thermodynamically large systems, where the interfaces between phases become negligible. Coexisting phases can thus be found by minimizing the average free energy density $\bar{f}$ of the entire system, which is given by

$$\bar{f}(N_\mathrm{P}, \{J_p\}, \{\phi_{p,i}\}) = \sum_{p=1}^{N_\mathrm{P}} J_p f(\{\phi_{p,i}\}) \,,$$

where N_C is the number of components, N_P is the number of phases, J_p denotes the fraction of volume that phase $p = 1, \ldots, N_\mathrm{P}$ occupies in the entire system, and $\phi_{p,i}$ is the volume fraction of component $i = 1, \ldots, N_\mathrm{C}$ in phase p. The physical behavior of the mixture is encoded in the free energy density f, which `flory` expresses using four orthogonal aspects: interaction, entropy, ensemble, and constraints. The package only imposes limits on the entropy part, which is crucial for the core algorithm, while the other three aspects are rather flexible. By combining these four aspects, `flory` supports a broad range of free energy densities f with different ensembles and constraints. A few widely-used specializations are provided for all four aspects, while customized ones can be added easily.

The `flory` package is designed to deliver high performance. The core part of the `flory` package is the finder for coexisting phases, which can be reused when the number N_C of components is kept fixed. This design moves a significant overhead to the creation of the solver, which can be amortized in many tasks, e.g., when a phase diagram is sampled. The core methods in the finder are just-in-time (JIT) compiled using numba (Lam et al., 2015) to achieve high performance. To support different forms of the free energy f, the core method is also designed to be general. The finder fetches compiled instances of the interaction, entropy, ensemble, and constraints, where case-specific codes are inserted as methods. These methods are also compiled for performance, using the experimental `jitclass` feature from numba (Lam et al., 2015).

The `flory` package adopts state-of-the-art numerical methods to determine coexisting phases. The main idea is to represent the system by many independent compartments, which can exchange particles and volumes, obeying total constraints (Zwicker & Laan, 2022). The `flory` package then minimizes the full free energy $\bar{f}$ instead of directly solving the corresponding coexistence conditions. At the free energy minimum, compartments may share similar compositions, which the package then cluster to obtain unique phases using the `scipy` cluster methods (Virtanen et al., 2020). This strategy circumvents the typical challenge of multiple local minima and it avoids iterating over all possible phase counts N_P. To improve performance, the `flory` package implements the improved Gibbs ensemble method we developed recently (Qiang et al., 2024). This method redistributes components across all compartments simultaneously, guided by a set of conjugate variables, such that the total computation cost per step only scales linearly with the number of compartments. Beside the core function of finding coexisting phases, `flory` also includes convenience tools, e.g., to analyze thermodynamic properties such as chemical potentials and osmotic pressures. In summary, the `flory` package can typically obtain the equilibrium coexisting states even in systems with many interacting components.

Examples

The following code illustrates the main functionality of the package by finding the two coexisting phases of a symmetric Flory-Huggins binary mixture with $\chi = 4$:

```
fh = flory.FloryHuggins(2, chis=[[0, 4.0], [4.0, 0]])
ensemble = flory.CanonicalEnsemble(2, phi_means=[0.5, 0.5])
finder = flory.CoexistingPhasesFinder(fh.interaction, fh.entropy, ensemble)
phases = finder.run().get_clusters()
```

Here, `FloryHuggins` represents the seminal Flory-Huggins free energy that creates the interaction `FloryHugginsInteraction` and the entropy `IdealGasEntropy` simultaneously, and provides tools for analyzing coexisting phases. By updating the interaction, we can then obtain the coexisting phases of another symmetric binary mixture with $\chi = 3.5$:

```
fh.chis = [[0, 3.5], [3.5, 0]]
finder.set_interaction(fh.interaction)
phases = finder.run().get_clusters()
```

This procedure can be repeated to sample an entire phase diagrams. Moreover, we could vary

the type of interaction by initializing a different class or modifying the existing one, and we could similarly change the entropy, ensemble, and constraints. Customized specialization of all four aspects can be easily implemented by deriving from the provided base classes.

Acknowledgements

We thank Chengjie Luo for stimulating discussions. We gratefully acknowledge funding from the Max Planck Society and the European Union (ERC, EmulSim, 101044662).

References

Jacobs, W. M. (2023). Theory and Simulation of Multiphase Coexistence in Biomolecular Mixtures. *J. Chem. Theory Comput.*, *19*(12), 3429–3445. https://doi.org/10.1021/acs.jctc.3c00198

Lam, S. K., Pitrou, A., & Seibert, S. (2015). Numba: A LLVM-based Python JIT compiler. *Proceedings of the Second Workshop on the LLVM Compiler Infrastructure in HPC*, 1–6. https://doi.org/10.1145/2833157.2833162

Lukas, H. L., Fries, S. G., & Sundman, B. (2007). *Computational thermodynamics: The Calphad method*. Cambridge Univ. Press. ISBN: 978-0-521-86811-2

Mao, S., Kuldinow, D., Haataja, M. P., & Košmrlj, A. (2019). Phase behavior and morphology of multicomponent liquid mixtures. *Soft Matter*, *15*(6), 1297–1311. https://doi.org/10.1039/C8SM02045K

Otis, R., & Liu, Z.-K. (2017). Pycalphad: CALPHAD-based Computational Thermodynamics in Python. *Journal of Open Research Software*, *5*(1). https://doi.org/10.5334/jors.140

Qiang, Y., Luo, C., & Zwicker, D. (2024). *Scaling of phase count in multicomponent liquids* (No. arXiv:2405.01138). arXiv. https://arxiv.org/abs/2405.01138

Shrinivas, K., & Brenner, M. P. (2021). Phase separation in fluids with many interacting components. *Proc. Natl. Acad. Sci. U.S.A.*, *118*(45), e2108551118. https://doi.org/10.1073/pnas.2108551118

Sundman, B., Lu, X.-G., & Ohtani, H. (2015). The implementation of an algorithm to calculate thermodynamic equilibria for multi-component systems with non-ideal phases in a free software. *Computational Materials Science*, *101*, 127–137. https://doi.org/10.1016/j.commatsci.2015.01.029

Virtanen, P., Gommers, R., Oliphant, T. E., Haberland, M., Reddy, T., Cournapeau, D., Burovski, E., Peterson, P., Weckesser, W., Bright, J., van der Walt, S. J., Brett, M., Wilson, J., Jarrod Millman, K., Mayorov, N., Nelson, A. R. J., Jones, E., Kern, R., Larson, E., … Contributors, S. 1. 0. (2020). SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python. *Nature Methods*.

Zwicker, D., & Laan, L. (2022). Evolved interactions stabilize many coexisting phases in multicomponent liquids. *Proc. Natl. Acad. Sci. U.S.A.*, *119*(28), e2201250119. https://doi.org/10.1073/pnas.2201250119

DRAFT